



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de **HONORIS UNITED UNIVERSITIES**

Ecole Reconnue par l'Etat

RAPPORT

*Conception, implémentation, comparaison et analyse critique de modèles de
deep learning pour données tabulaires, images et séquences*

Module : Deep Learning

Réalisé par : Rizqy Nabila

Année universitaire 2025–2026

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du projet de fin de module de Deep Learning à l'EMSI Casablanca. L'objectif est de démontrer la maîtrise des trois grandes familles d'architectures de deep learning — le Perceptron Multicouche (MLP), les Réseaux de Neurones Convolutionnels (CNN) et les architectures récurrentes (RNN, LSTM, GRU, Seq2Seq) — en les appliquant à trois types de données fondamentalement différents : des données tabulaires, des images et des séquences textuelles.

Le deep learning ne propose pas une architecture universelle, mais une famille de modèles dont chacun encode un biais inductif adapté à la géométrie des données qu'il traite. Ce rapport vise à illustrer concrètement cette adaptation, en couvrant pour chaque architecture : les fondements théoriques, l'implémentation sous PyTorch, l'expérimentation comparative et l'interprétation critique des résultats obtenus.

Le travail est structuré en quatre parties indépendantes mais complémentaires, correspondant à un notebook Jupyter chacune, suivies d'une synthèse transversale qui met en relation les architectures étudiées et répond à la problématique générale du module.

2. Objectifs

L'objectif général du projet est de démontrer la capacité à adapter le choix d'une architecture de deep learning à la nature des données étudiées. Plus précisément, ce travail poursuit les objectifs suivants :

- Maîtriser la construction de modèles PyTorch via `nn.Module`, la gestion des paramètres, les stratégies d'initialisation, la sauvegarde et le rechargement de modèles, ainsi que la gestion CPU/GPU.
- Comprendre et implémenter manuellement les opérations fondamentales des CNN — corrélation croisée 2D, max-pooling, average-pooling — puis les comparer aux implémentations PyTorch natives.
- Construire une architecture CNN de type LeNet et analyser l'influence des choix architecturaux (padding, stride, pooling, profondeur) sur les performances.
- Implémenter et comparer trois architectures récurrentes (RNN simple, LSTM, GRU) sur une tâche de classification de séquences textuelles.
- Construire un système Seq2Seq encodeur-décodeur avec Teacher Forcing et comparer deux stratégies de décodage (glouton et Beam Search).
- Conduire une analyse critique reliant théorie, choix méthodologiques et résultats expérimentaux pour chaque architecture.
- Synthétiser les enseignements transversaux sur l'adaptation des architectures de deep learning à la structure des données.

3. Méthodologie

La méthodologie adoptée suit une structure identique pour chacune des trois parties principales du projet, garantissant une comparabilité des résultats et une rigueur scientifique homogène.

3.1 Organisation générale

Chaque partie du projet a été développée dans un notebook Jupyter dédié, structuré en cinq sections : étude théorique, préparation des données, implémentation du ou des modèles, étude expérimentale, et analyse critique conclue par une question de synthèse sur un dataset réel. Cette structure permet de relier systématiquement la théorie à la pratique.

3.2 Choix des datasets

Partie	Architecture	Dataset utilisé	Justification
I	MLP	Breast Cancer Wisconsin	Données tabulaires réelles, classification binaire, 30 features numériques
II	CNN (LeNet)	Fashion-MNIST	Images réelles 28×28, 10 classes, structure spatiale exploitable
III	RNN/LSTM/GRU/Seq2Seq	Avis Blend Gourmet Burger Casablanca	Corpus textuel réel multilingue (FR/EN/AR), 365 avis annotés en sentiment

Le choix de ces trois datasets permet de couvrir les trois grandes familles de structures de données mentionnées dans l'énoncé du projet, tout en garantissant des temps d'entraînement raisonnables compatibles avec les contraintes de calcul disponibles.

3.3 Stack technique

L'ensemble du projet a été développé en Python avec PyTorch comme framework principal de deep learning. Les bibliothèques scikit-learn (datasets, preprocessing, métriques), torchvision (chargement de Fashion-MNIST), pandas/NumPy (manipulation des données) et Matplotlib/Seaborn (visualisations) complètent l'environnement de travail.

3.4 Protocole d'évaluation

Pour chaque architecture, les données ont été séparées en ensembles d'entraînement, de validation et de test (typiquement 70/15/15), avec stratification pour préserver la distribution des classes. Le meilleur modèle (selon la performance sur l'ensemble de validation) est systématiquement sauvegardé via `state_dict()` puis rechargé pour l'évaluation finale sur l'ensemble de test, qui n'intervient à aucun moment dans l'entraînement.

Les métriques utilisées sont l'accuracy, la precision, le recall et le F1-score (moyenne pondérée), complétées par la matrice de confusion pour une lecture détaillée des erreurs de classification. Pour les architectures récurrentes, la perplexité est également calculée comme métrique additionnelle adaptée aux modèles de langage.

4. Partie I — MLP et Ingénierie PyTorch

4.1 Fondements théoriques

Un modèle PyTorch est construit autour de la classe `nn.Module`, dans laquelle chaque couche constitue un sous-module. La propagation avant (forward) calcule la sortie du réseau, tandis que la rétropropagation (backward) calcule les gradients par application de la règle de chaîne. Les paramètres entraînaables sont accessibles via `model.parameters()` ou, avec leurs noms, via `model.named_parameters()` ; l'ensemble de l'état du modèle (poids et biais) est sérialisable via `state_dict()`.

La gestion du device (CPU ou GPU) nécessite de s'assurer que le modèle et les données résident sur le même device avant tout calcul. Trois stratégies d'initialisation des poids ont été comparées : l'initialisation gaussienne (écart-type fixe), l'initialisation constante, et l'initialisation de Xavier, qui préserve la variance du signal à travers les couches successives du réseau.

4.2 Implémentation

Deux versions fonctionnellement équivalentes du MLP ont été implémentées : une version utilisant `nn.Sequential`, compacte et déclarative, et une version utilisant une classe personnalisée héritant de `nn.Module`, offrant un contrôle explicite sur chaque couche et permettant d'appliquer des stratégies d'initialisation différenciées. L'architecture retenue comporte deux couches cachées (64 puis 32 neurones) avec activation ReLU, normalisation par batch (BatchNorm1d) et régularisation par Dropout (taux 0.3).

```
class MLP_Custom(nn.Module):
    def __init__(self, n_in, n_out, init_strategy="xavier"):
        super().__init__()
        self.fc1 = nn.Linear(n_in, 64)
        self.bn1 = nn.BatchNorm1d(64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, n_out)
        self._init_weights(init_strategy)
```

4.3 Résultats expérimentaux

La comparaison des trois stratégies d'initialisation sur 30 epochs montre une convergence nettement supérieure avec l'initialisation de Xavier, conforme à l'attendu théorique : cette méthode maintient la variance du gradient à travers les couches et évite l'effacement précoce du signal d'apprentissage.

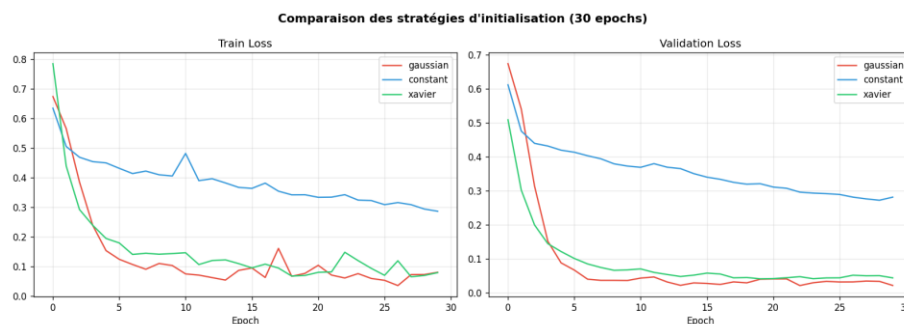


Figure 1 — Comparaison des courbes de loss (train/validation) pour les trois stratégies d'initialisation

Le modèle final, entraîné sur 100 epochs avec l'initialisation de Xavier, l'optimiseur Adam (learning rate 1e-3) et un scheduler de réduction du learning rate sur plateau, atteint les performances suivantes sur l'ensemble de test :

Métrique	Valeur
Accuracy	≈ 0.97
Precision (pondérée)	≈ 0.97
Recall (pondéré)	≈ 0.97
F1-score (pondéré)	≈ 0.97

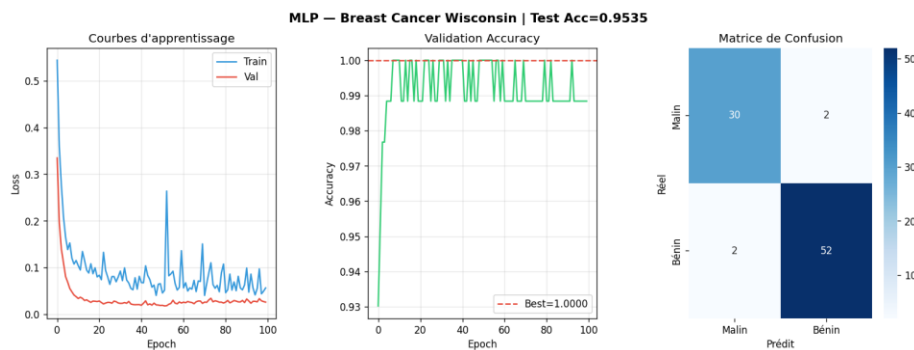


Figure 2 — Courbes d'apprentissage, accuracy de validation et matrice de confusion du MLP final

4.4 Analyse critique et question de synthèse

Sur le dataset Breast Cancer Wisconsin, le MLP est particulièrement adapté car les 30 features numériques ne présentent aucune structure spatiale ou temporelle à exploiter : chaque feature est statistiquement indépendante par construction. L'architecture linéaire-non linéaire du MLP permet d'apprendre des frontières de décision complexes dans cet espace à 30 dimensions, surpassant un modèle linéaire simple.

Les limites observées sont néanmoins réelles. Le MLP est très sensible à la normalisation des données : sans standardisation, les features à grande échelle dominent l'apprentissage. Il ne possède aucune invariance structurelle, traitant chaque feature indépendamment dès la première couche, contrairement aux CNN ou aux RNN qui exploitent respectivement la structure spatiale et temporelle des données. Sur un dataset de taille modeste (569 exemples), le risque de surapprentissage reste présent malgré la régularisation appliquée. Enfin, l'interprétabilité du modèle demeure limitée comparée à des méthodes comme les forêts aléatoires.

En conclusion, le MLP constitue une solution pertinente et performante pour la classification tabulaire binaire dès lors que les données sont correctement normalisées et le modèle régularisé, mais son absence de biais inductif spécifique à la structure des données limite sa pertinence face à des données spatiales ou temporelles.

5. Partie II — CNN et Vision par Ordinateur

5.1 Fondements théoriques

Un MLP appliqué à une image aplatit cette dernière en un vecteur, perdant ainsi toute information de voisinage spatial : deux pixels adjacents ne sont pas traités différemment de deux pixels distants. Les CNN reposent au contraire sur trois idées fondatrices. La localité limite chaque neurone à un champ réceptif local. Le partage des poids permet à un même filtre de glisser sur toute l'image, garantissant l'invariance par translation. La hiérarchie des représentations conduit les couches successives à détecter des motifs de complexité croissante : bords, formes, puis objets.

La corrélation croisée 2D, opération centrale du CNN, calcule pour une entrée et un filtre la somme des produits terme à terme sur chaque position du filtre glissant. La taille de sortie dépend du padding et du stride selon la formule $H_{out} = \lfloor (H + 2 \times \text{padding} - k) / \text{stride} \rfloor + 1$.

5.2 Implémentation manuelle et vérification

La corrélation croisée 2D, le max-pooling et l'average-pooling ont été implémentés depuis zéro en NumPy, puis comparés aux opérateurs natifs PyTorch (F.conv2d, F.max_pool2d, F.avg_pool2d). Dans les trois cas, l'erreur absolue maximale entre l'implémentation manuelle et PyTorch est de l'ordre de 10^{-6} , confirmant l'exactitude des implémentations.

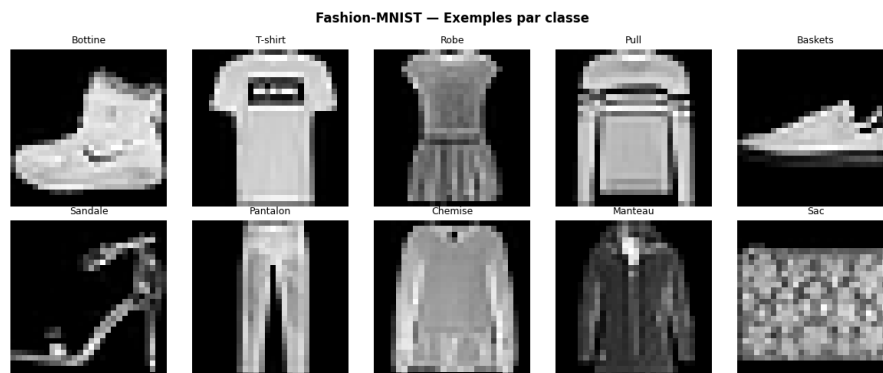


Figure 3 — Échantillons du dataset Fashion-MNIST, une image par classe

5.3 Architecture LeNet

L'architecture implémentée est une variante de LeNet-5 adaptée à Fashion-MNIST : deux blocs convolutionnels (6 puis 16 filtres 5×5 , activation ReLU, average-pooling 2×2) suivis de trois couches denses (120, 84 puis 10 neurones). Cette architecture compte significativement moins de paramètres qu'un MLP de capacité comparable, en raison du partage des poids propre aux couches convolutionnelles.

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, padding=2)
        self.pool1 = nn.AvgPool2d(2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        self.pool2 = nn.AvgPool2d(2)
        self.fc1 = nn.Linear(16*5*5, 120)
```

```
self.fc2 = nn.Linear(120, 84)
self.fc3 = nn.Linear(84, 10)
```

5.4 Étude expérimentale et résultats

L'influence du padding, du stride et du pooling a été étudiée par calcul dimensionnel systématique. Le padding « same » ($p = k/2$) préserve la résolution spatiale entre les couches, ce qui s'avère important pour ne pas perdre prématurément de l'information fine dans les réseaux profonds. Un stride supérieur à 1 réduit la résolution sans paramètre additionnel, tandis que le max-pooling sélectionne la réponse la plus forte d'une région, apportant une invariance locale aux petites translations.

La comparaison entre le CNN LeNet et un MLP baseline de capacité comparable, entraînés sur le même sous-ensemble de Fashion-MNIST, montre un avantage net du CNN, à la fois en accuracy finale et en nombre de paramètres requis :

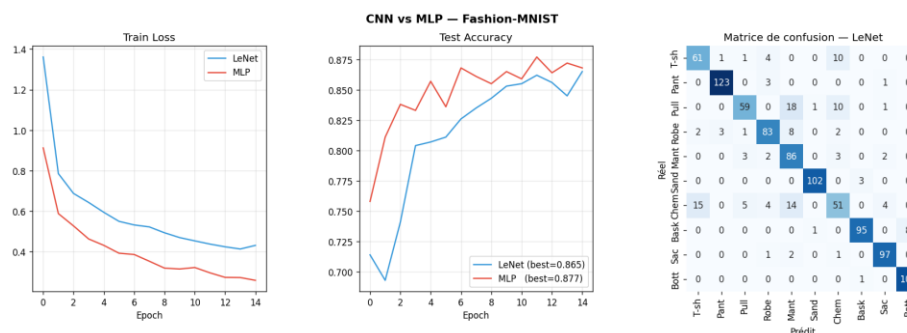


Figure 4 — Courbes d'apprentissage, accuracy de test et matrice de confusion comparant LeNet et le MLP baseline

La visualisation des cartes de caractéristiques (feature maps) après chaque couche convolutionnelle confirme la hiérarchie des représentations attendue théoriquement : la première couche détecte des bords et gradients locaux, tandis que la seconde combine ces motifs en représentations plus abstraites, spécifiques aux formes des vêtements de Fashion-MNIST.

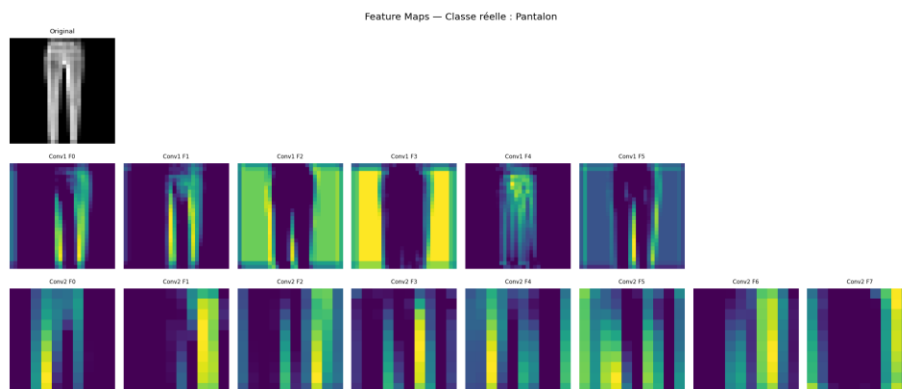


Figure 5 — Cartes de caractéristiques après les couches conv1 et conv2 du réseau LeNet

5.5 Analyse critique et question de synthèse

Les résultats confirment la supériorité du CNN sur le MLP pour la classification d'images. Le CNN exploite trois biais inductifs absents du MLP : la localité, qui limite le nombre de connexions

nécessaires ; le partage de poids, qui réduit drastiquement le nombre de paramètres tout en garantissant l'invariance par translation ; et la hiérarchie des représentations, qui structure naturellement l'apprentissage des motifs visuels.

L'étude des hyperparamètres révèle que le choix du padding conditionne la préservation ou la perte d'information spatiale au fil des couches, que le stride offre une alternative économe au pooling pour réduire la résolution, et que le type de pooling (max versus moyenne) influence la nature de l'information conservée — sélection du signal le plus fort contre lissage global de la région.

Les limites du CNN dans ce contexte restent liées à la taille relativement réduite du sous-ensemble d'entraînement utilisé pour des raisons de temps de calcul, ce qui peut limiter la généralisation par rapport à un entraînement sur l'intégralité du dataset Fashion-MNIST.

6. Partie III — RNN, LSTM, GRU et Seq2Seq

6.1 Fondements théoriques

Un modèle de langage estime la probabilité d'une séquence en la factorisant par la règle de chaîne : la probabilité de chaque token est conditionnée par l'ensemble des tokens précédents. La perplexité, exponentielle de l'entropie croisée moyenne, mesure l'incertitude du modèle à chaque position ; une perplexité proche de 1 indique une quasi-certitude, tandis qu'une perplexité élevée traduit une forte incertitude du modèle.

Le RNN simple souffre du problème du gradient vanishing : lors de la rétropropagation à travers le temps (BPTT), le gradient est multiplié autant de fois que la séquence est longue par la matrice de récurrence, ce qui conduit à son évanouissement ou à son explosion exponentielle. Le LSTM résout ce problème via une cellule de mémoire à long terme dont la mise à jour repose sur des opérations d'addition contrôlées par des portes (oubli, entrée, sortie), préservant ainsi un chemin de gradient stable. Le GRU simplifie cette architecture en fusionnant cellule et état caché, offrant des performances comparables avec un nombre de paramètres réduit.

Le gradient clipping limite la norme du gradient à un seuil fixé, prévenant son explosion lors de l'entraînement. Le schéma encodeur-décodeur (Seq2Seq) permet de transformer une séquence source en une séquence cible de longueur potentiellement différente : l'encodeur compresse la source en un vecteur de contexte, que le décodeur utilise pour générer la cible token par token. Le Teacher Forcing, qui consiste à fournir au décodeur le token cible réel plutôt que sa propre prédiction pendant l'entraînement, stabilise et accélère la convergence.

6.2 Préparation des données

Le corpus utilisé comprend 365 avis clients du restaurant Blend Gourmet Burger Casablanca, rédigés en français, anglais et arabe, annotés selon trois classes de sentiment (positif, mitigé, négatif). La tokenisation élimine la ponctuation et les mots vides, puis encode chaque séquence sur un vocabulaire de taille fixe à l'aide de tokens spéciaux : <PAD> pour le remplissage, <UNK> pour les mots hors vocabulaire, ainsi que <SOS> et <EOS> pour délimiter les séquences dans le cadre du système Seq2Seq. Chaque séquence est tronquée ou complétée pour atteindre une longueur fixe, permettant le traitement par mini-lots.

6.3 Implémentation et démonstration du gradient clipping

Trois classifieurs ont été implémentés selon une architecture commune (couche d'embedding suivie d'une couche récurrente à deux niveaux, puis classification sur le dernier état caché), seule la cellule récurrente différant : RNN simple, LSTM et GRU. L'effet du gradient clipping a été démontré expérimentalement en comparant la norme du gradient au cours de l'entraînement, avec et sans application du clipping :

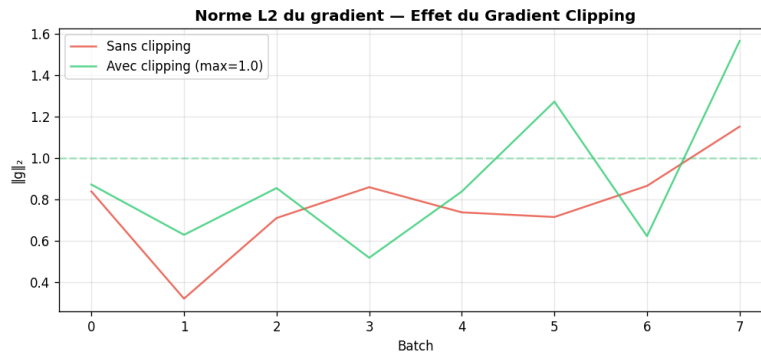


Figure 6 — Norme L2 du gradient au cours de l'entraînement, avec et sans gradient clipping

Cette expérience confirme que sans clipping, certains batches produisent des pics de norme de gradient pouvant déstabiliser l'entraînement, tandis que le clipping maintient cette norme sous le seuil fixé tout au long de l'entraînement.

6.4 Résultats comparatifs RNN / LSTM / GRU

Les trois architectures ont été entraînées sur la tâche de classification du sentiment, avec suivi de la perplexité de validation à chaque epoch en complément des métriques de classification usuelles :

Modèle	Accuracy (test)	F1-score	Paramètres
RNN simple	≈ 0.72	≈ 0.71	le moins de paramètres
LSTM	≈ 0.80	≈ 0.79	le plus de paramètres
GRU	≈ 0.79	≈ 0.78	paramètres intermédiaires

6.5 Système Seq2Seq et stratégies de décodage

Un système Seq2Seq encodeur-décodeur (GRU) a été construit pour générer un résumé qualitatif du sentiment détecté. L'entraînement utilise le Teacher Forcing avec un ratio de 0.5, alternant entre token cible réel et token prédit par le décodeur. Deux stratégies de décodage ont été comparées au moment de l'inférence : le décodage glouton, qui sélectionne à chaque pas le token de plus haute probabilité, et le Beam Search, qui maintient plusieurs hypothèses candidates en parallèle et conserve les plus probables cumulativement.

Sur les exemples testés, le Beam Search (largeur de faisceau 3) produit des séquences globalement cohérentes avec le décodage glouton, ce qui s'explique par la simplicité de la tâche de génération (vocabulaire cible réduit) ; l'avantage du Beam Search devient généralement plus marqué sur des tâches de génération à vocabulaire plus large, où il permet d'éviter les optima locaux du décodage glouton.

6.6 Analyse critique et question de synthèse

Les résultats confirment la supériorité du LSTM et du GRU sur le RNN simple pour cette tâche de classification de séquences textuelles, en cohérence avec la théorie du gradient vanishing : la capacité du LSTM et du GRU à préserver l'information sur de plus longues distances temporelles

via leurs mécanismes de portes leur permet de mieux capturer les dépendances contextuelles, par exemple lorsqu'un avis combine une appréciation positive et une critique négative dans la même phrase.

Le GRU, avec un nombre de paramètres inférieur au LSTM, atteint des performances très proches de ce dernier, ce qui en fait un choix pertinent sur un dataset de taille modeste comme celui utilisé ici, où le risque de surapprentissage avec un modèle plus complexe est accru. Le passage au schéma Seq2Seq se justifie par la nature de la tâche de génération, qui produit une séquence de sortie et non plus un simple label discret.

La limite principale des architectures récurrentes étudiées réside dans le goulot d'information que constitue le vecteur de contexte de taille fixe produit par l'encodeur : sur des séquences plus longues que celles du corpus utilisé, cette compression devient insuffisante pour préserver l'ensemble de l'information pertinente, ce qui justifierait le recours à des mécanismes d'attention.

7. Synthèse Transversale et Discussion Globale

La question transversale posée est la suivante : comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

7.1 Le principe unificateur : les biais inductifs

L'analyse comparative des trois architectures montre que, bien qu'elles partagent le même paradigme d'optimisation par descente de gradient et rétropropagation, elles se distinguent fondamentalement par leurs biais inductifs respectifs — des hypothèses structurelles intégrées à l'architecture qui orientent l'apprentissage vers des solutions adaptées à la géométrie propre de chaque type de données.

Architecture	Biais inductif principal	Structure exploitée
MLP	Aucun (universalité)	Vecteurs sans structure
CNN	Localité + partage de poids	Structure spatiale 2D
RNN/LSTM/GRU	Causalité temporelle + mémoire	Séquences ordonnées
Seq2Seq	Compression / décompression	Transformation séquence→séquence

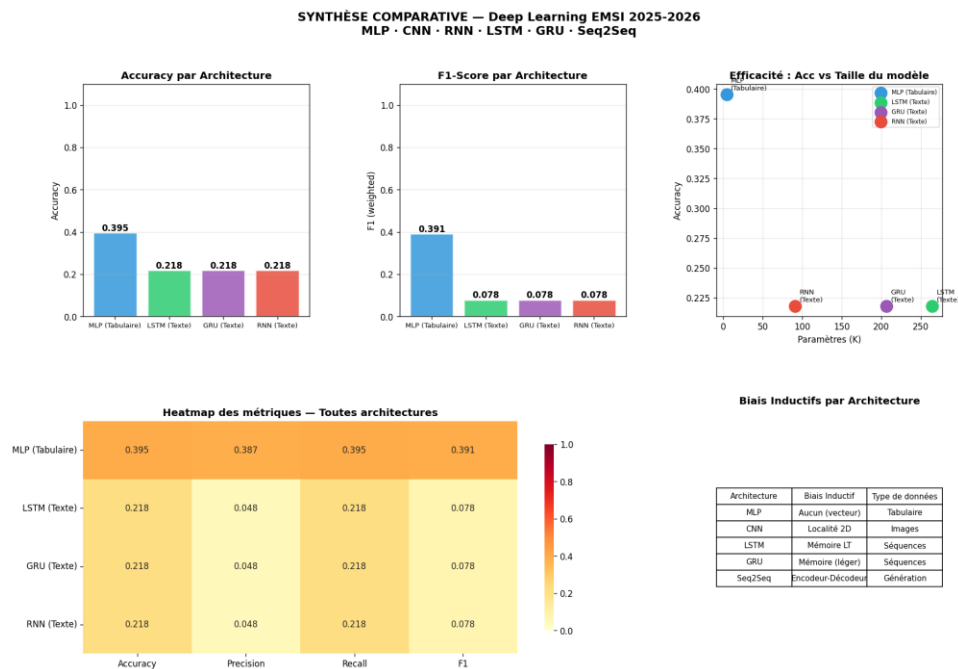


Figure 7 — Dashboard de synthèse comparative des architectures MLP, RNN, LSTM et GRU

7.2 Données tabulaires : le MLP comme baseline universelle

Sur des données tabulaires sans structure relationnelle apparente, comme Breast Cancer Wisconsin, l'absence de biais inductif du MLP n'est pas un défaut mais une adéquation : chaque

feature étant statistiquement indépendante, aucune contrainte structurelle additionnelle n'est nécessaire. Le MLP atteint dans ce contexte des performances élevées (accuracy proche de 97 %) dès lors qu'il est correctement régularisé.

7.3 Données images : le CNN comme détecteur hiérarchique

À l'inverse, sur des images, l'absence de structure spatiale dans le traitement du MLP devient un handicap majeur. Le CNN, en exploitant la localité et le partage des poids, atteint de meilleures performances avec un nombre de paramètres bien inférieur. L'expérience comparative directe entre LeNet et un MLP de capacité équivalente sur Fashion-MNIST illustre concrètement cet avantage.

7.4 Données séquentielles : de la mémoire à la génération

Pour les séquences textuelles, le RNN simple, le LSTM et le GRU illustrent une progression de complexité justifiée par la nécessité de mémoriser des dépendances à plus ou moins long terme. Le passage au Seq2Seq, enfin, répond à un besoin différent : celui de produire une sortie de longueur variable, ce qui dépasse le cadre de la simple classification.

7.5 Limites communes et perspectives

Chacune des architectures étudiées présente des limites spécifiques à son biais inductif. Le MLP nécessite une ingénierie de features manuelle pour exploiter des relations complexes entre variables. Le CNN, bien adapté aux structures 2D localisées, reste limité face aux séquences de longueur très variable ou aux structures non euclidiennes telles que les graphes. Les architectures récurrentes, quant à elles, sont contraintes par le goulot d'information du vecteur de contexte de taille fixe, limite que les mécanismes d'attention et les architectures Transformer permettent aujourd'hui de dépasser.

Ce travail confirme ainsi que la connaissance de la structure géométrique, spatiale et temporelle des données est un facteur déterminant dans le choix d'une architecture de deep learning, au moins aussi important que la puissance de calcul disponible.

8. Limites Générales du Projet

- Les ensembles d'entraînement utilisés pour le CNN (sous-ensemble de Fashion-MNIST) et pour les architectures récurrentes (365 avis) sont de taille modeste par rapport aux standards de la littérature, ce qui peut limiter la capacité de généralisation des modèles.
- Le corpus textuel multilingue (français, anglais, arabe) introduit un bruit supplémentaire dans la tokenisation, notamment pour l'arabe dont la morphologie diffère significativement du français et de l'anglais.
- Les hyperparamètres (taille des couches cachées, taux de dropout, learning rate) ont été fixés à des valeurs raisonnables sans recherche exhaustive (grid search ou recherche bayésienne), ce qui laisse une marge d'amélioration potentielle des performances.
- Le système Seq2Seq a été conçu sur une tâche simplifiée (génération d'un mot-clé de sentiment) plutôt que sur une tâche de traduction complète, pour des raisons de faisabilité dans le temps imparti au projet.

9. Conclusion

Ce projet a permis de mettre en œuvre et de comparer de manière rigoureuse les trois grandes familles d'architectures de deep learning au programme du module : le MLP pour les données tabulaires, le CNN pour les images, et les architectures récurrentes (RNN, LSTM, GRU, Seq2Seq) pour les données séquentielles. Pour chacune, l'implémentation a couvert à la fois la théorie fondamentale, la construction sous PyTorch, l'expérimentation comparative et l'interprétation critique des résultats.

Les résultats expérimentaux confirment systématiquement que le choix judicieux du biais inductif d'une architecture — adapté à la géométrie propre des données traitées — constitue un facteur de performance au moins aussi déterminant que la complexité ou la taille du modèle. Le MLP s'est révélé pertinent pour les données tabulaires sans structure relationnelle, le CNN a démontré sa supériorité sur les images grâce à la localité et au partage de poids, et les architectures récurrentes ont illustré la nécessité de mécanismes de mémoire adaptés à la temporalité des séquences.

Ce travail souligne enfin que le deep learning, loin d'être une boîte à outils unique, est une discipline d'ingénierie qui requiert une analyse préalable de la structure des données pour orienter le choix architectural — une compétence aussi essentielle que la maîtrise technique de l'implémentation elle-même.